

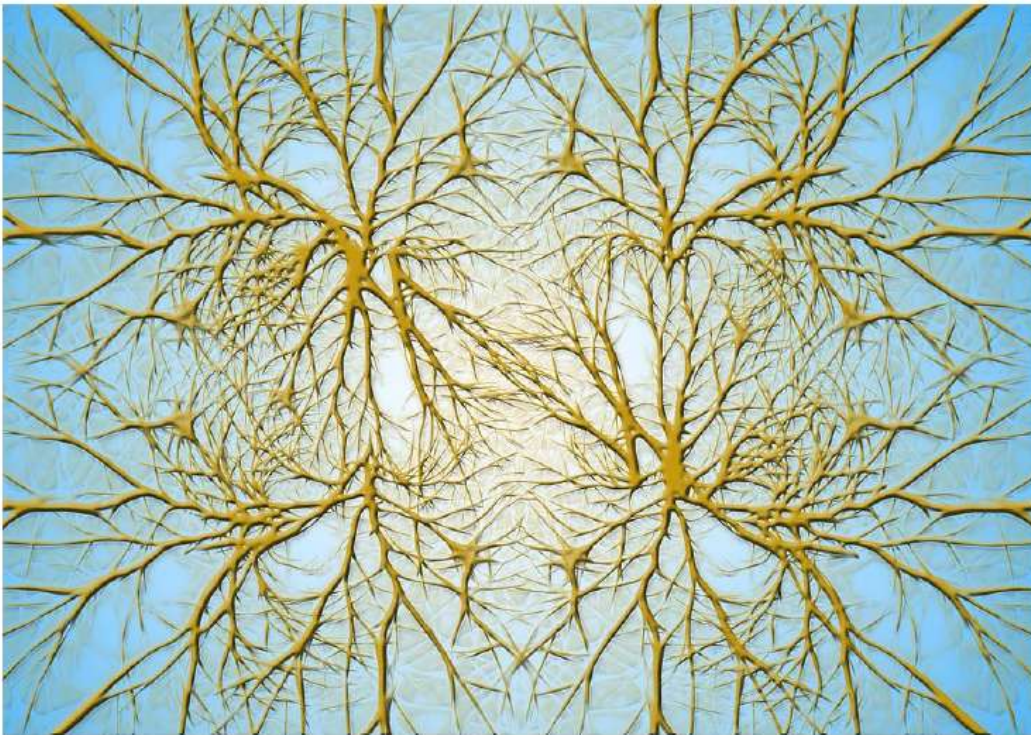
ANN : Multi Layered Perceptron (MLP)

Sourav Karmakar

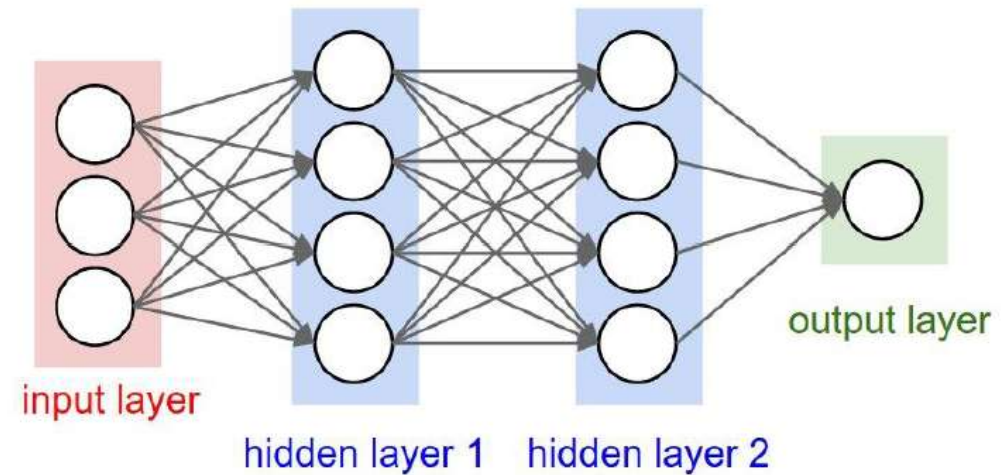
souravkarmakar29@gmail.com

Artificial Neural Networks

Biological Neurons:
Complex connectivity patterns

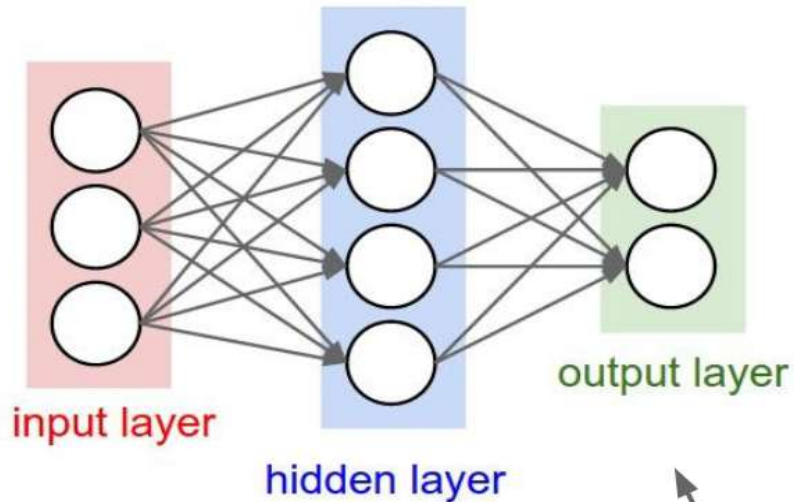


Neurons in a neural network:
Organized into regular layers for
computational efficiency

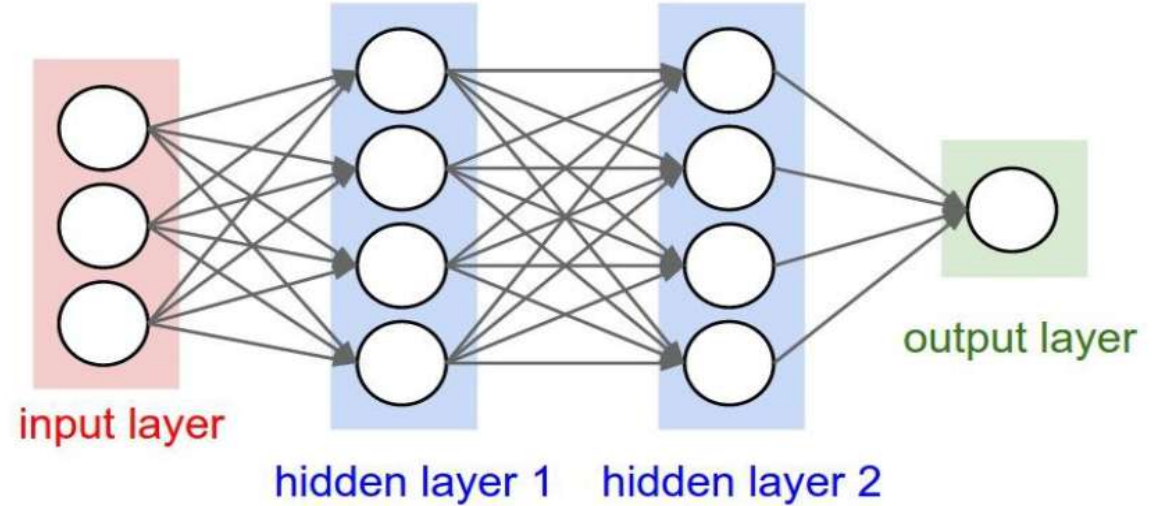


This type of architectures are called **Multi-Layered Perceptron (MLP)**

Multi Layered Perceptron (MLP)



“2-layer Neural Net”, or
“1-hidden-layer Neural Net”



“3-layer Neural Net”, or
“2-hidden-layer Neural Net”

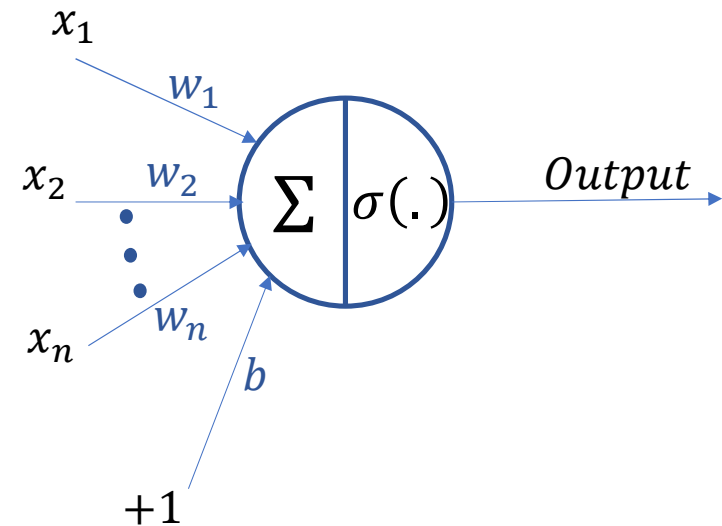
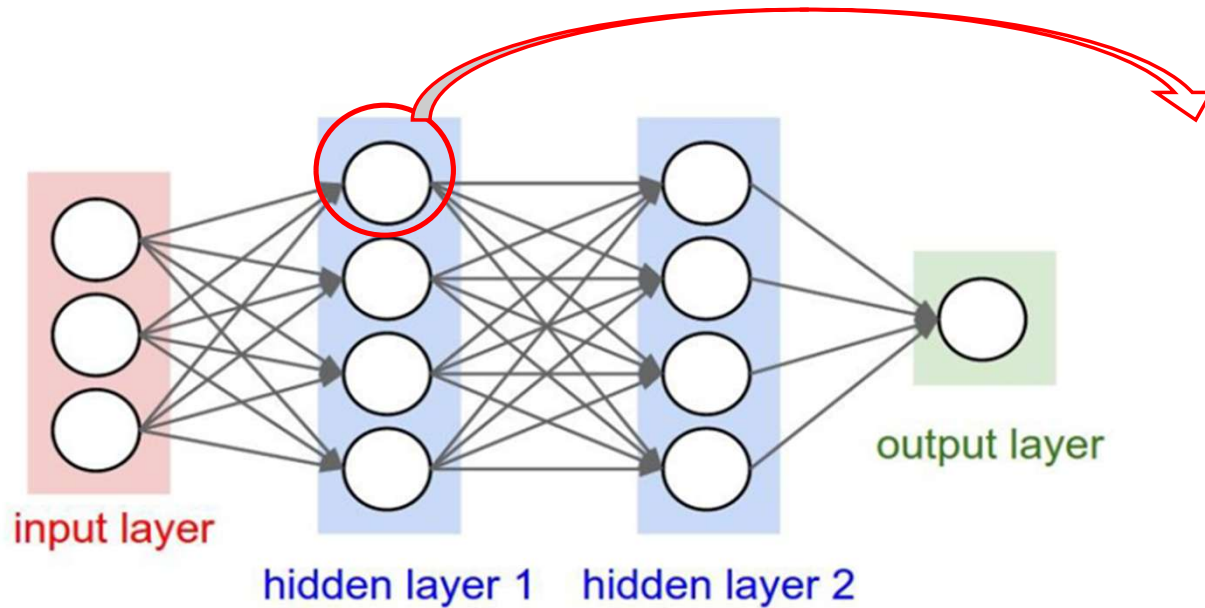
“**Fully-connected**” layers

Fully-connected layers are also called **Dense Layers**

The standard convention in Neural Network is to count only the layers that have learnable weights (the layers where computation occurs). So, people don't consider input layer as a layer because it has no computation in it. Hence, the first diagram will be called “2-layer Neural Net” and the second one “3-layer Neural Net”.

Multi Layered Perceptron (MLP)

A closed look at a neuron in the Neural Network



Each node in the computation layer connected to every node of previous layer.

$$\begin{aligned} \text{Output} &= \sigma(w_1x_1 + w_2x_2 + \dots + w_nx_n + b) \\ &= \sigma(\mathbf{w}^T \mathbf{x} + b) \end{aligned}$$

$\sigma(\cdot)$ is the activation function, which is a non-polynomial and continuous function.

The Role of Activation Functions

Without Activation Function:

- Suppose we only use **linear operations** in a network: $y = Wx + b$
- Even if we stack multiple layers, the output is still just a **linear transformation** of the input.
$$y = W_3(W_2(W_1x + b_1) + b_2) + b_3 = W'x + b'$$
- That means no matter how many layers you add, the whole network is just a linear model. Hence, without activation function the model can't capture the complex non-linear relationship in the data.

Purpose of Activation Function:

1. Introduce Non-linearity:

- Helps capture complex patterns in data by introducing non-linearity.
- With nonlinear activations, neural networks become **universal function approximators** (proved by the **Universal Approximation Theorem**)

2. Control Information Flow:

- Decide which neurons "fire" (activate) and which don't.
- Makes the network selective about what information passes forward.

3. Bounded Outputs / Normalization:

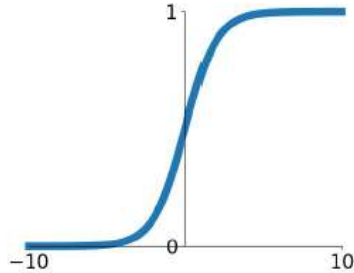
- Some activations (like sigmoid, tanh) squash outputs into a small range, making training stable.

4. Gradient Flow: Differentiable activation functions allow gradient-based learning (backpropagation)

Different Activation Functions

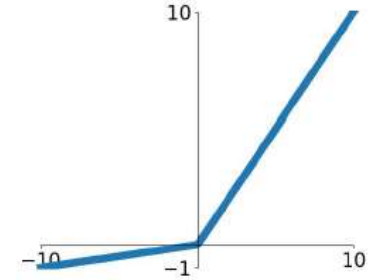
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



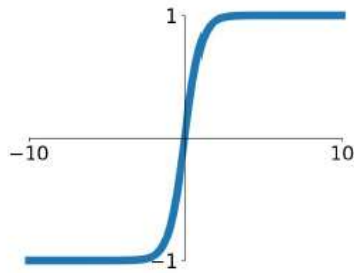
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

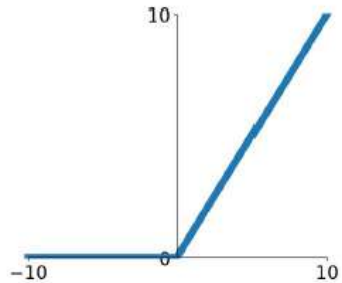


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

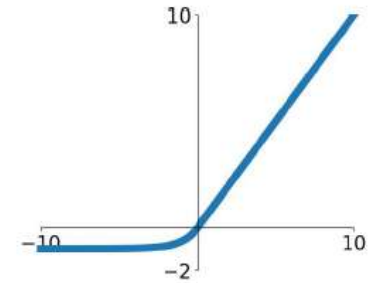
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



ReLU (Rectified Linear Unit) is a good practical choice of activation function in hidden layers.

Softmax Activation Function

Softmax Activation is a generalization of *Sigmoid function*, and useful for multiclass classification problems.

- Consider the three-class problem. The final output using softmax activation is computed as following:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \xrightarrow{\hspace{1cm}} \begin{bmatrix} e^{z_1} \\ e^{z_2} \\ e^{z_3} \end{bmatrix} \xrightarrow{\hspace{1cm}} \frac{1}{e^{z_1} + e^{z_2} + e^{z_3}} \begin{bmatrix} e^{z_1} \\ e^{z_2} \\ e^{z_3} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

Pre-activation
($z_i \in \mathbb{R}$)

Exponentiation
($e^{z_i} \in \mathbb{R}^+$)

Normalization
 $\left(y_i \in (0,1) \text{ and } \sum y_i = 1 \right)$

For k class problem:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_k \end{bmatrix} = \frac{1}{\sum_{i=1}^k e^{z_i}} \begin{bmatrix} e^{z_1} \\ e^{z_2} \\ \vdots \\ e^{z_k} \end{bmatrix}$$

- Softmax activation normalizes the outputs for each class to fall between 0 and 1. Also after softmax the sum of the output = 1. Which means softmax activation creates a **legitimate probability distribution** over classes.
- We choose the most probable class (by argmax function) as the final prediction for multi-class classification.

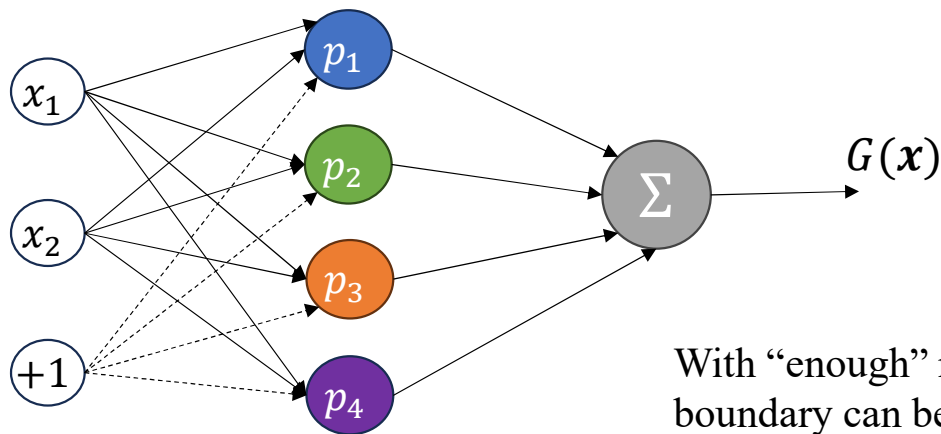
Universal Approximation Theorem

- **The proposition:** A feed-forward neural network with a *single hidden layer* containing a *finite number of neurons* can approximate any continuous function to any desired degree of accuracy.
- **The requirement:**
 - **Non-linearity:** The activation function (σ) must be non-linear (like: sigmoid, relu etc.)
 - **Width:** The hidden layer must have “sufficient” neurons.
- **Intuition:** Think of Neural Network as **universal curve fitter**. No matter how complex the original function $f(\mathbf{x})$ is, we can construct a neural network $G(\mathbf{x})$ that mimics it.

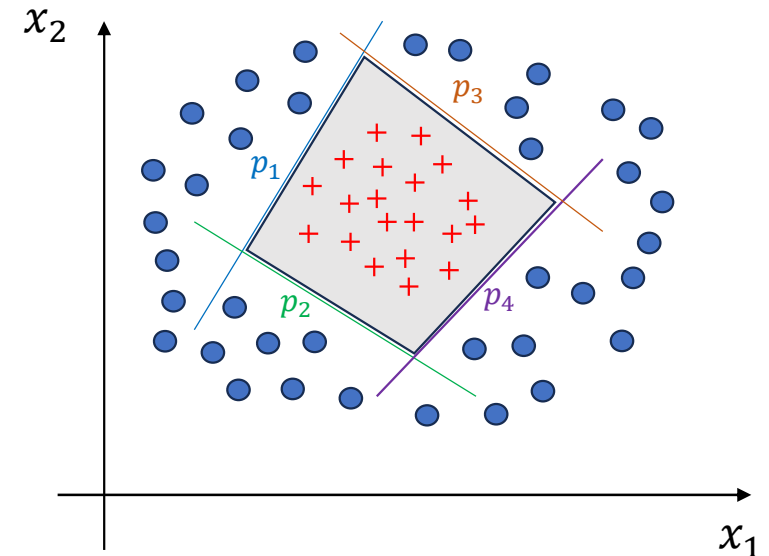
Consider the non-linearly separable data as shown beside.

Each node (perceptron) will produce a linear boundary.

When combined they will be able to produce the complex boundary.



With “enough” number of neurons in the hidden layer, any complex shape boundary can be produced.



Universal Approximation Theorem

Mathematical Formulation:

- Let, $f(\mathbf{x})$ is a continuous function, where $\mathbf{x} \in \mathbb{R}^n$.
- We can construct a single-hidden-layer neural network with N neurons and its output be:

$$G(\mathbf{x}) = \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i)$$

Where:

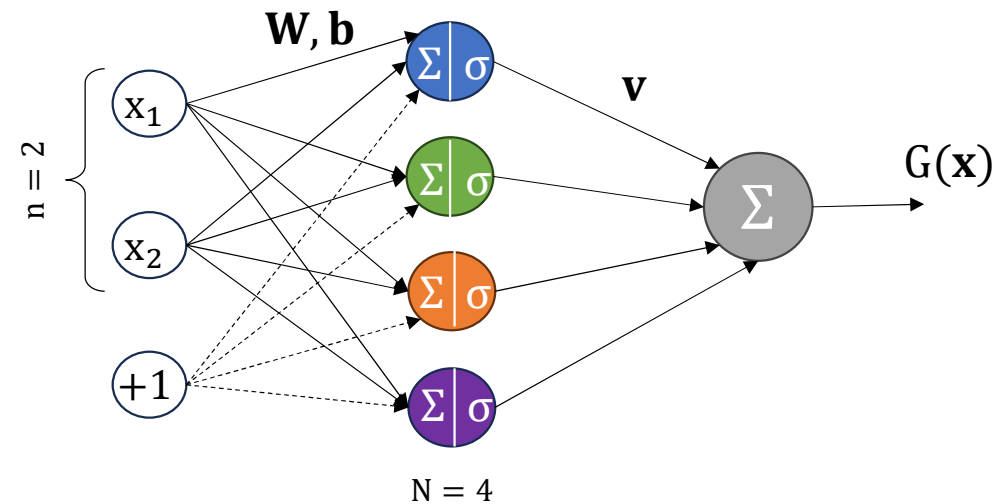
- $\mathbf{x} \in \mathbb{R}^n$: input vector
- $\mathbf{w}_i$: Weights connecting input to hidden neuron i
- b_i : Bias of hidden neuron i
- v_i : Weights connecting hidden neuron i to the output
- σ : A non-polynomial and continuous activation function

The Theorem States:

For any $\epsilon > 0$, there exists an N such that:

$$|G(\mathbf{x}) - f(\mathbf{x})| < \epsilon$$

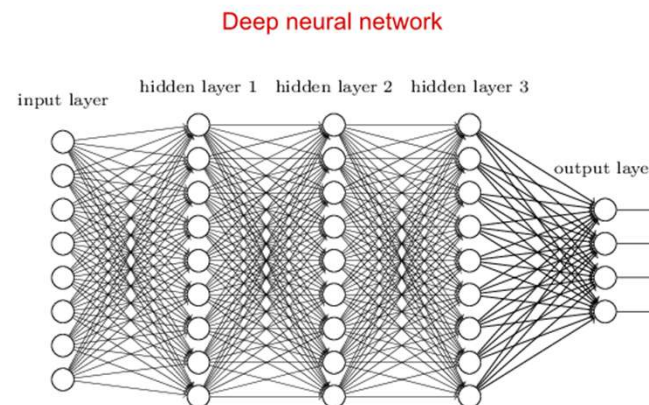
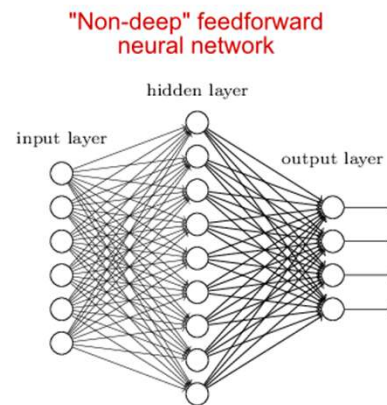
This means the error ϵ can be made arbitrarily small by increasing the number of neurons in the hidden layer N .



Shallow vs Deep Neural Network

The Caveat of UAT (The “Width” crisis) : Theoretical Guarantee vs. Practical Reality

- **The “Unknown N ”:** The Universal Approximation Theorem proves that a hidden layer *exists*, but it provides no upper bound on the number of neurons (N).
- **The Exponential Explosion:** To approximate a highly complex, “wiggly” function in high-dimensional space, a shallow network may require an **exponential number of neurons**.
 - *Result:* The model becomes a “Memory Table” rather than a “Learner”. It tries to memorize every point instead of understanding the underlying logic (overfitting).
- **The Efficiency Gap:**
 - **Shallow/Wide:** High parameter count → High risk of Overfitting → Massive Memory Requirement.
 - **Deep/Narrow:** Can achieve the same (or better) approximation with a fraction of the parameters.



Shallow vs Deep Neural Network

Why Deep is better than Wide: The power of Depth and Feature Hierarchy

1. **Compositionality:** Deep networks break a complex problem into a hierarchy of simpler sub-problems.

For example, in case of image recognition:

- Layer-1 learns Edges.
- Layer-2 learns Shapes from those edges.
- Layer-3 learns Objects from those shapes.

UAT perspective: Instead of one layer trying to fit a complex curve all at once, each layer “bends” the space slightly for the next layer

2. **Automatic feature extraction:** In traditional ML, humans hand-engineer features. In Deep Learning:
 - The early layers act as Feature Extractors.
 - The final layer act as the classifier.
 - The network decides which characteristics of the input vector $\mathbf{x}$ are actually relevant for the output $G(\mathbf{x})$.

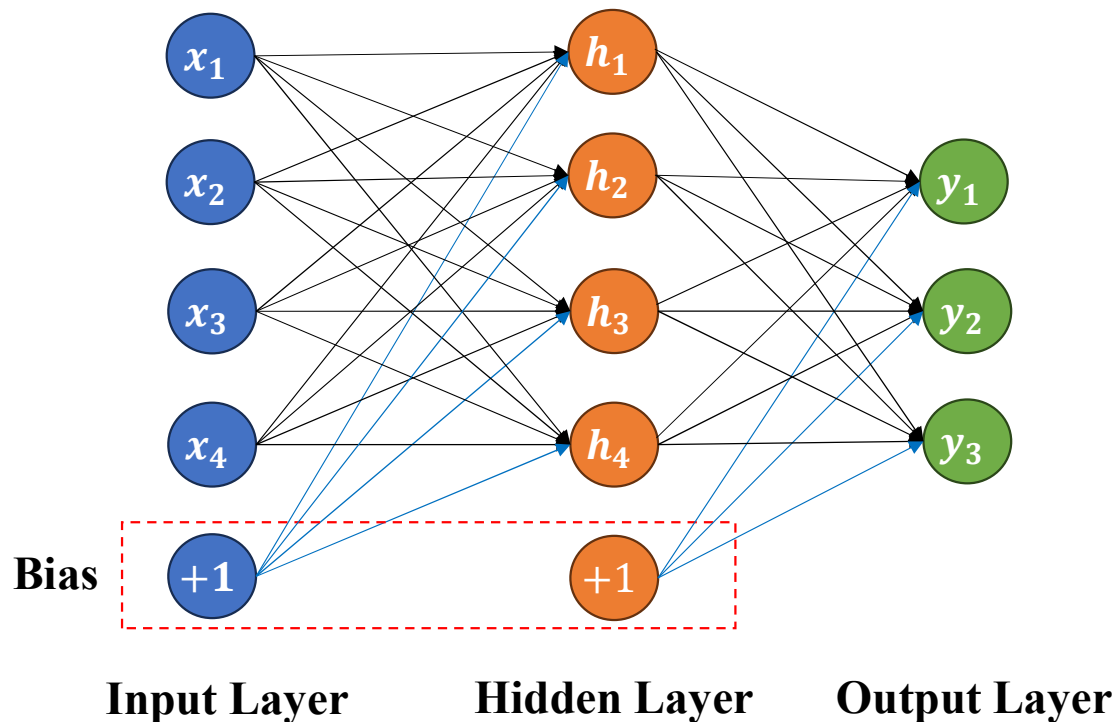
Multi Layered Perceptron (MLP): Architecture

- A layer of sensory nodes called input nodes and the layer is called input layer. No computation happens in input nodes.
- The number of nodes in the input layer = number of features or attributes of the data.
- Layers of computing nodes. These are called **hidden layers**. In successive layers, the nodes are all connected by learnable weights. This is called ‘dense architecture’.
- Number of Hidden layers and Number of nodes in each hidden layer are user choice (hyper-parameters) and there is no general guideline for the choice of number of hidden layers or the number of nodes in each hidden layers. However, there should be at least one hidden layer.
- Layer of final computing nodes is called output layer.
- The number of nodes in the output layer = number of classes of the dataset. (for a classification problem). For multi-class classification it is usually called ‘softmax’ layer due to the softmax activation. For binary classification only one output node with sigmoid activation is needed. For regression problem, there will be only one output node without any activation function.
- Usually we use ‘**ReLU**’ activation for all nodes in hidden layer and ‘**softmax**’ activation in output layer for multi-class classification problem or ‘**sigmoid**’ activation in output layer for binary classification problem. However, we can also use different activation functions in the hidden layer.

Multi Layered Perceptron (MLP): Architecture

For example: To classify the simple IRIS dataset using ANN

- The number of input nodes = 4 (as the dataset has four features / attributes)
- The number of output nodes = 3 (as there are three classes)
- We can build an ANN with one hidden layer containing 4 nodes (say)



- Number of connections from input layer to hidden layer:

$$(4 + 1) \times 4 = 20$$

Number of input nodes Bias Number of nodes in hidden layer

- Number of connections from hidden layer to output layer:

$$(4 + 1) \times 3 = 15$$

Number of nodes in hidden layer Bias Number of nodes in output layer

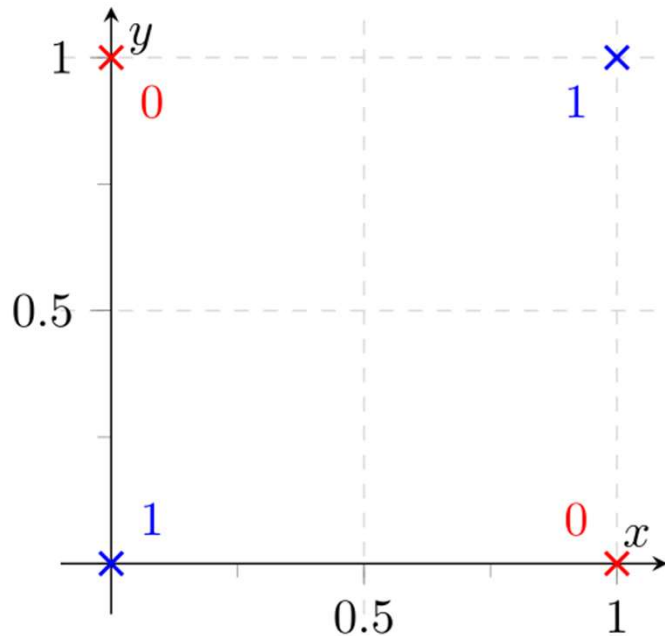
Multi Layered Perceptron (MLP): Advantages

- **Non-linearity:** MLP is very suitable to model a physical phenomenon which is in general non-linear.
- **Automatic Feature Extraction:** Each layer in MLP learns a representation of the data. The network learns features automatically during training by adjusting weights. The learned features are optimal for final task as MLP is an universal function approximator. The first few hidden layers learn low level patterns, where as the deeper layers learn high-level representation. This is called **representation learning**.
- **Adaptivity:** It can cope with the change in dataset distribution.
- **Parallelism:** It can be implemented in multi-core parallel architecture (like GPU) for faster computation.
- **Robustness:** It has inherent ability to handle confusing / noisy / missing datapoints
- **Fault tolerance:** It has the ability to work to some extent even in case of component (neuron) failure.
- **Complex decision boundary:** It has the ability to have a complex decision boundary for classification.

We shall use MLP with supervised learning for complex classification tasks.

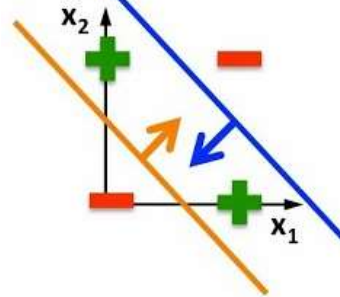
How MLP Solved The XOR Problem

XOR Problem

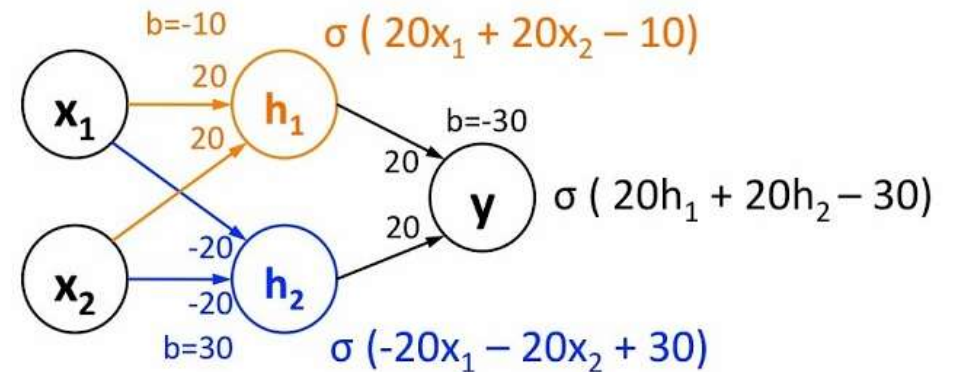


Input 1	Input 2	Output
0	0	0
0	1	1
1	1	0
1	0	1

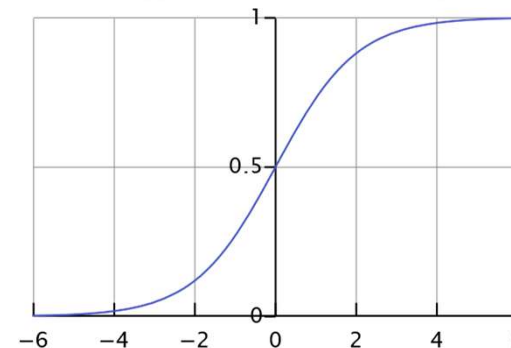
Linear classifiers cannot solve this



Multi Layered Perceptron

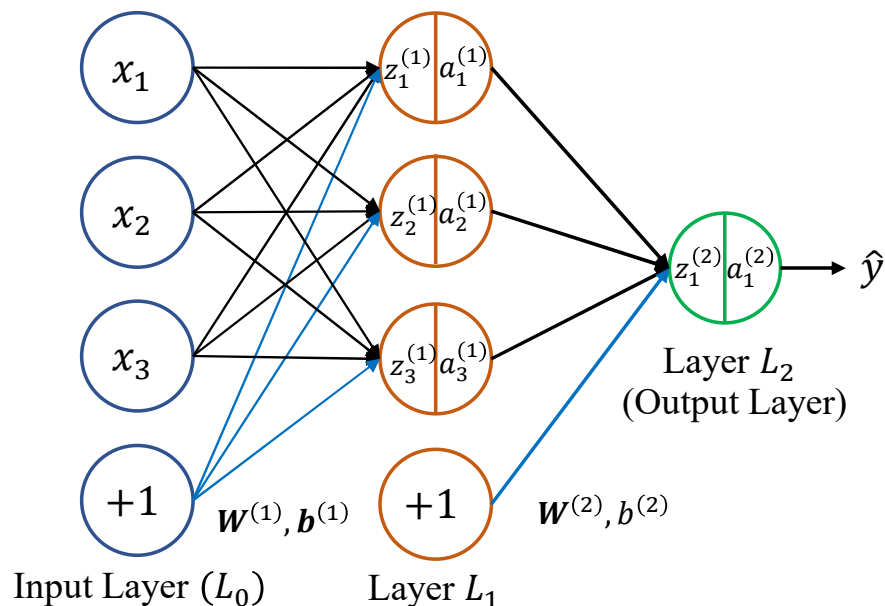


$$\begin{array}{lll}
 \sigma(20 \cdot 0 + 20 \cdot 0 - 10) \approx 0 & \sigma(-20 \cdot 0 - 20 \cdot 0 + 30) \approx 1 & \sigma(20 \cdot 0 + 20 \cdot 1 - 30) \approx 0 \\
 \sigma(20 \cdot 1 + 20 \cdot 1 - 10) \approx 1 & \sigma(-20 \cdot 1 - 20 \cdot 1 + 30) \approx 0 & \sigma(20 \cdot 1 + 20 \cdot 0 - 30) \approx 0 \\
 \sigma(20 \cdot 0 + 20 \cdot 1 - 10) \approx 1 & \sigma(-20 \cdot 0 - 20 \cdot 1 + 30) \approx 1 & \sigma(20 \cdot 1 + 20 \cdot 1 - 30) \approx 1 \\
 \sigma(20 \cdot 1 + 20 \cdot 0 - 10) \approx 1 & \sigma(-20 \cdot 1 - 20 \cdot 0 + 30) \approx 1 & \sigma(20 \cdot 1 + 20 \cdot 1 - 30) \approx 1
 \end{array}$$



Multi Layered Perceptron (MLP)

Key Notations:



- n_l denotes the number of computing layers in our network; thus in the figure beside $n_l = 2$.
- s_l denotes the number of nodes in layer l (not counting the bias unit).
- L_0 is the input layer and L_{n_l} is the output layer.
- $b_i^{(l)}$ is the bias associated with unit i in layer l .

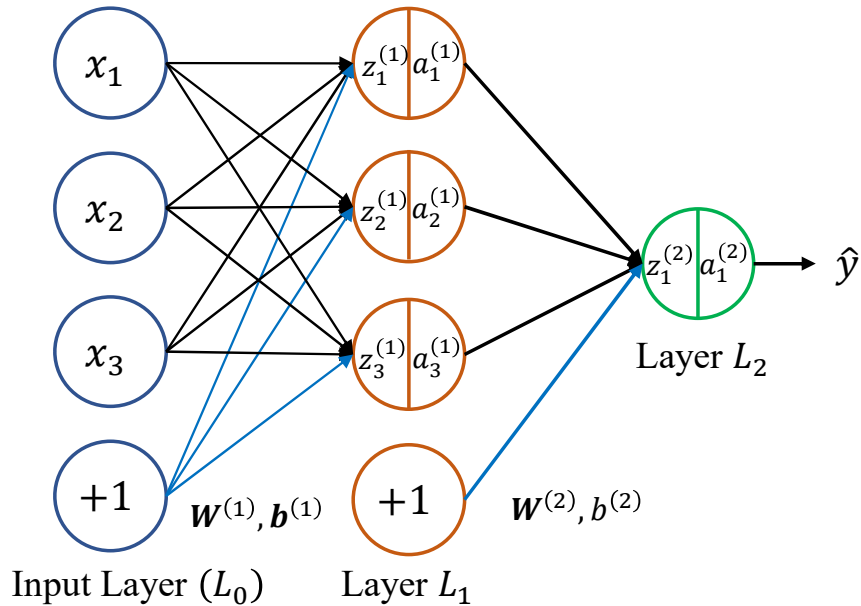
- The neural network as shown above has parameters $\langle \mathbf{W}, \mathbf{b} \rangle = \langle (\mathbf{W}^{(1)}, \mathbf{b}^{(1)}), (\mathbf{W}^{(2)}, \mathbf{b}^{(2)}) \rangle$,

Where $w_{ij}^{(l)}$ denotes the parameter (or weight) associated with the connection between unit j in layer $l - 1$, and unit i in layer l . Note that bias units don't have inputs or connections going into them, since they always output value $+1$.

- In this example we have $\mathbf{W}^{(1)} \in \mathbb{R}^{3 \times 3}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{3 \times 1}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{1 \times 3}$, and $\mathbf{b}^{(2)} \in \mathbb{R}$

Multi Layered Perceptron (MLP)

Forward Propagation:



- $z_i^{(l)}$ is the pre-activation of unit i in layer l . This is the weighted summation of inputs connected to that unit.
- $a_i^{(l)}$ to denote the post activation output of unit i in layer l . For $l = 0$, we also use $a_i^{(0)} = x_i$ to denote i^{th} input.

$$a_1^{(1)} = f(z_1^{(1)}) = f(w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + w_{13}^{(1)}x_3 + b_1^{(1)})$$

$$a_2^{(1)} = f(z_2^{(1)}) = f(w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{23}^{(1)}x_3 + b_2^{(1)})$$

$$a_3^{(1)} = f(z_3^{(1)}) = f(w_{31}^{(1)}x_1 + w_{32}^{(1)}x_2 + w_{33}^{(1)}x_3 + b_3^{(1)})$$

Here $f(\cdot)$ denotes the activation function.

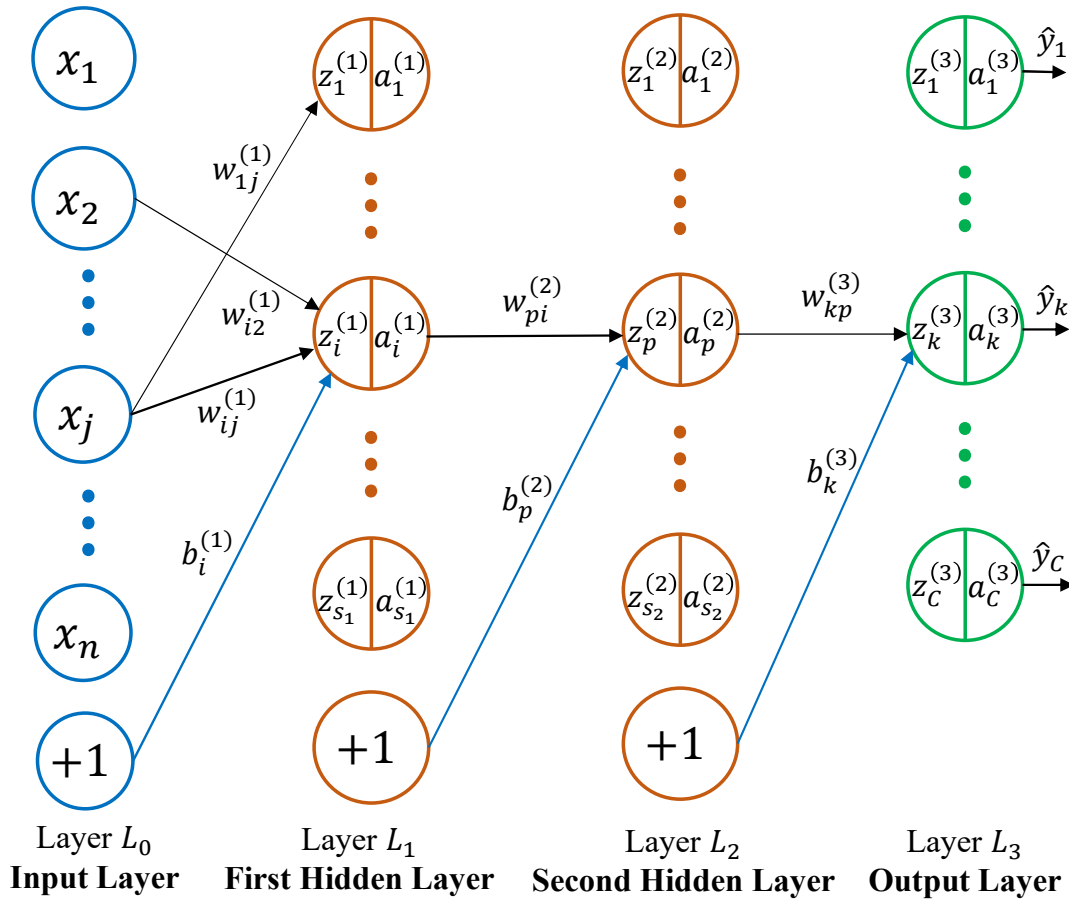
$$\text{The final output: } \hat{y} = a_1^{(2)} = f(z_1^{(2)}) = f(w_{11}^{(2)}a_1^{(1)} + w_{12}^{(2)}a_2^{(1)} + w_{13}^{(2)}a_3^{(1)} + b_1^{(2)})$$

All the equations shown above can be written in simplified fashion in vector matrix notation:

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) = f(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

Multi Layered Perceptron (MLP)

- n_l denotes the number of layers in the network. s_l denotes the number of nodes in layer l .
- $w_{ij}^{(l)}$ denotes the parameter (or weight) associated with the connection between unit j in layer $l - 1$, and unit i in layer l .
- $b_i^{(l)}$ is the bias associated with unit i in layer l .



$$\begin{bmatrix} z_1^{(1)} \\ \vdots \\ z_i^{(1)} \\ \vdots \\ z_{s_1}^{(1)} \end{bmatrix}_{s_2 \times 1} = \begin{bmatrix} w_{11}^{(1)} & \dots & w_{1n}^{(1)} \\ \vdots & \ddots & \vdots \\ w_{i1}^{(1)} & \dots & w_{in}^{(1)} \\ \vdots & \ddots & \vdots \\ w_{s_1 1}^{(1)} & \dots & w_{s_1 n}^{(1)} \end{bmatrix}_{s_1 \times n} \begin{bmatrix} x_1 \\ \vdots \\ x_j \\ \vdots \\ x_n \end{bmatrix}_{n \times 1} + \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_i^{(1)} \\ \vdots \\ b_{s_1}^{(1)} \end{bmatrix}_{s_1 \times 1} \quad \text{and} \quad \begin{bmatrix} a_1^{(1)} \\ \vdots \\ a_i^{(1)} \\ \vdots \\ a_{s_1}^{(1)} \end{bmatrix}_{s_1 \times 1} = f \left(\begin{bmatrix} z_1^{(1)} \\ \vdots \\ z_i^{(1)} \\ \vdots \\ z_{s_1}^{(1)} \end{bmatrix}_{s_1 \times 1} \right)$$

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \quad \text{and} \quad \mathbf{a}^{(1)} = f(\mathbf{z}^{(1)})$$

$$\begin{bmatrix} z_1^{(2)} \\ \vdots \\ z_p^{(2)} \\ \vdots \\ z_{s_2}^{(2)} \end{bmatrix}_{s_2 \times 1} = \begin{bmatrix} w_{11}^{(2)} & \dots & w_{1s_1}^{(2)} \\ \vdots & \ddots & \vdots \\ w_{p1}^{(2)} & \dots & w_{ps_1}^{(2)} \\ \vdots & \ddots & \vdots \\ w_{s_2 1}^{(2)} & \dots & w_{s_2 s_1}^{(2)} \end{bmatrix}_{s_2 \times s_1} \begin{bmatrix} a_1^{(1)} \\ \vdots \\ a_i^{(1)} \\ \vdots \\ a_{s_1}^{(1)} \end{bmatrix}_{s_1 \times 1} + \begin{bmatrix} b_1^{(2)} \\ \vdots \\ b_p^{(2)} \\ \vdots \\ b_{s_2}^{(2)} \end{bmatrix}_{s_2 \times 1} \quad \text{and} \quad \begin{bmatrix} a_1^{(2)} \\ \vdots \\ a_p^{(2)} \\ \vdots \\ a_{s_2}^{(2)} \end{bmatrix}_{s_2 \times 1} = f \left(\begin{bmatrix} z_1^{(2)} \\ \vdots \\ z_p^{(2)} \\ \vdots \\ z_{s_2}^{(2)} \end{bmatrix}_{s_2 \times 1} \right)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} \quad \text{and} \quad \mathbf{a}^{(2)} = f(\mathbf{z}^{(2)})$$

$$\begin{bmatrix} z_1^{(3)} \\ \vdots \\ z_k^{(3)} \\ \vdots \\ z_c^{(3)} \end{bmatrix}_{c \times 1} = \begin{bmatrix} w_{11}^{(3)} & \dots & w_{1s_2}^{(3)} \\ \vdots & \ddots & \vdots \\ w_{k1}^{(3)} & \dots & w_{ks_2}^{(3)} \\ \vdots & \ddots & \vdots \\ w_{c1}^{(3)} & \dots & w_{cs_2}^{(3)} \end{bmatrix}_{c \times s_2} \begin{bmatrix} a_1^{(2)} \\ \vdots \\ a_p^{(2)} \\ \vdots \\ a_{s_2}^{(2)} \end{bmatrix}_{s_2 \times 1} + \begin{bmatrix} b_1^{(3)} \\ \vdots \\ b_k^{(3)} \\ \vdots \\ b_c^{(3)} \end{bmatrix}_{c \times 1} \quad \text{and} \quad \begin{bmatrix} a_1^{(3)} \\ \vdots \\ a_k^{(3)} \\ \vdots \\ a_c^{(3)} \end{bmatrix}_{c \times 1} = f \left(\begin{bmatrix} z_1^{(3)} \\ \vdots \\ z_k^{(3)} \\ \vdots \\ z_c^{(3)} \end{bmatrix}_{c \times 1} \right)$$

$$\mathbf{z}^{(3)} = \mathbf{W}^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)} \quad \text{and} \quad \mathbf{a}^{(3)} = f(\mathbf{z}^{(3)})$$

The predicted output, $\hat{\mathbf{y}} = \mathbf{a}^{(3)}$

Multi Layered Perceptron (MLP)

Loss / Cost Function:

Given a training set of 'm' samples, we can define the overall cost functions in following way:

- **Mean Squared Error Loss:** Suitable for regression tasks.

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{m} \left[\sum_{i=1}^m \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2 \right]$$

- **Binary Cross-entropy Loss:** Suitable for binary classification.

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

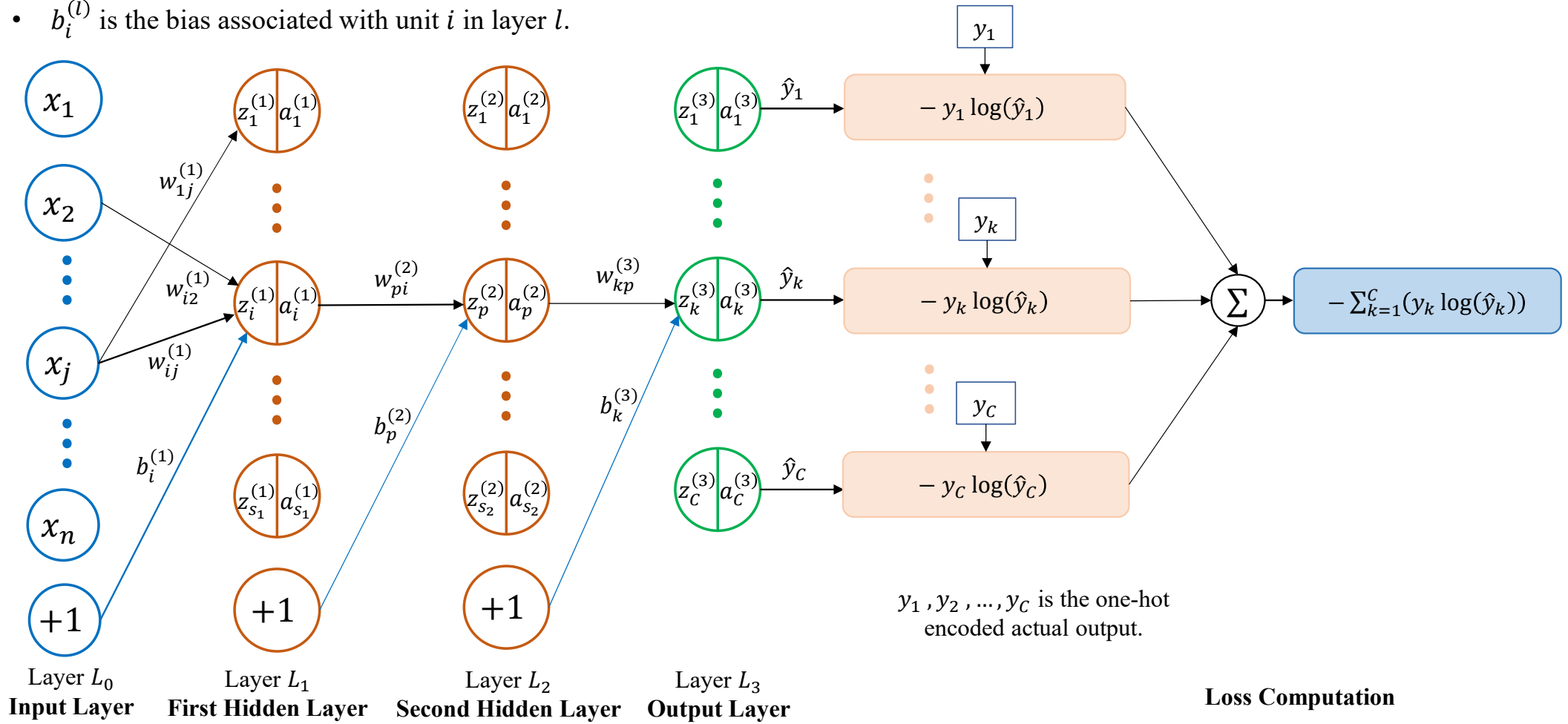
- **Categorical Cross-entropy Loss:** Suitable for multi-class classification (number of classes = C).

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^C y_k^{(i)} \log(\hat{y}_k^{(i)}) \right]$$

$\hat{y}^{(i)}$ is the model's prediction for input $x^{(i)}$ and the actual label is $y^{(i)}$.

Multi Layered Perceptron (MLP)

- n_l denotes the number of layers in the network. s_l denotes the number of nodes in layer l .
- $w_{ij}^{(l)}$ denotes the parameter (or weight) associated with the connection between unit j in layer $l - 1$, and unit i in layer l .
- $b_i^{(l)}$ is the bias associated with unit i in layer l .



Multi Layered Perceptron (MLP)

Back Propagation Learning Algorithm:

The weights and biases in the neural network are updated using Backpropagation Algorithm, where each weights are updated using Gradient Descent update rule (or some variant of it). For that we have to calculate the partial derivatives of the loss function with respect to the weights and biases.

User has to specify:

- Number of **computation layers**: n_l , or people often specify number of hidden layers which is $n_l - 1$.
- Number of **nodes** in each hidden layer: s_l for $l = 1, 2, \dots, n_l - 1$
- Number of **epochs** for which the network to be trained. An epoch = one complete pass of the entire dataset through the neural network. During an epoch, the model sees all training samples once and update weights depending on the **batch size**.
- The **loss function** to be used by the neural network.
- The optimization technique (also called **optimizer**) to be used to optimize the loss. The most basic optimizer is gradient descent.
- The **batch size**: The number of training samples processed before the model updates its weights. (discussed in the next slide)

Gradient Descent Optimization Technique

General idea of Gradient Descent:

We want to minimize a loss function $J(\mathbf{W}, \mathbf{b})$ with respect to model parameters $\mathbf{W}$ and $\mathbf{b}$.

Update rule: (showing only for w for simplification)

$$w \leftarrow w - \eta \nabla_w J ; \eta \Rightarrow \text{learning rate}, \nabla_w J \Rightarrow \text{gradient of loss } J \text{ w.r.t. parameter } w.$$

(a) Batch Gradient Descent:

- In each epoch the error in the final layer is average of errors due to all the training samples in the dataset.
- Update the parameters / weights using backpropagation after all the training samples are fed to the network.
- This is stable and achieves smooth convergence. However, this is very slow for large dataset.

(b) Stochastic Gradient Descent (SGD):

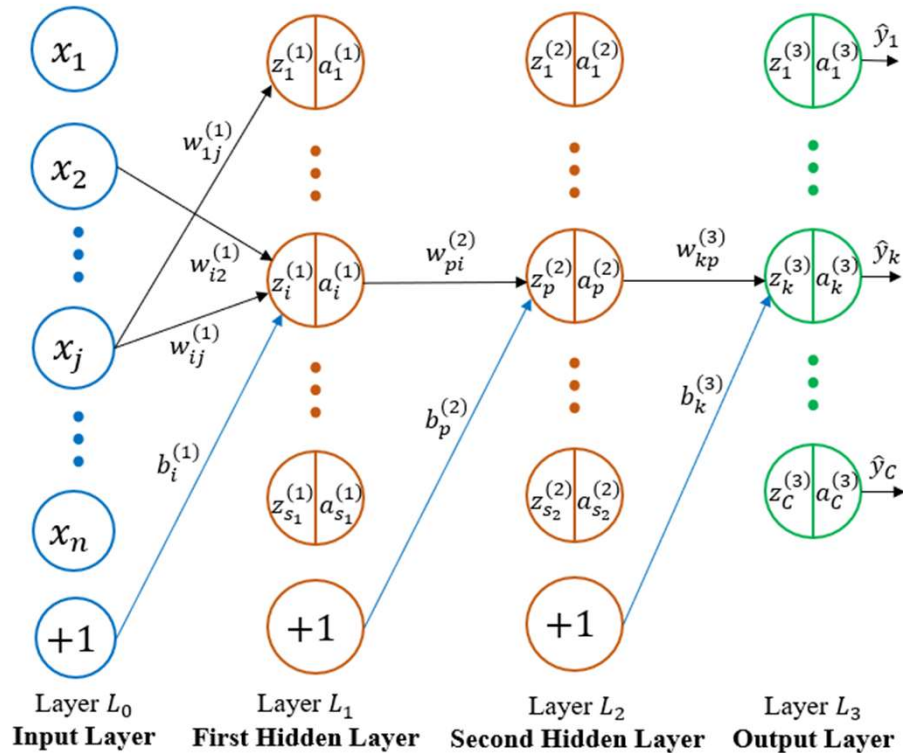
- Updates the weight after each training sample.
- Faster convergence but noisy updates due to randomness.
- It can escape local minima due to randomness.

(c) Mini-batch Gradient Descent:

- Uses small random subsets (mini-batches) of data to compute gradients. Compromise between batch and SGD.
- For example: if dataset size = 5000 and (mini)batch size = 100, then one epoch = 50 batches. Each (mini)batch produces one weight update, so 50 iterations per epoch.
- It is most widely used in practice. The (mini)batch size depends on dataset size, GPU/RAM memory, and training stability. Most commonly used (mini)batch sizes are 32, 64, 128, 256 etc. [i.e. between 32 to 256]

Computation Graph

To better understand the flow of forward-pass computations, let us draw a computation graph

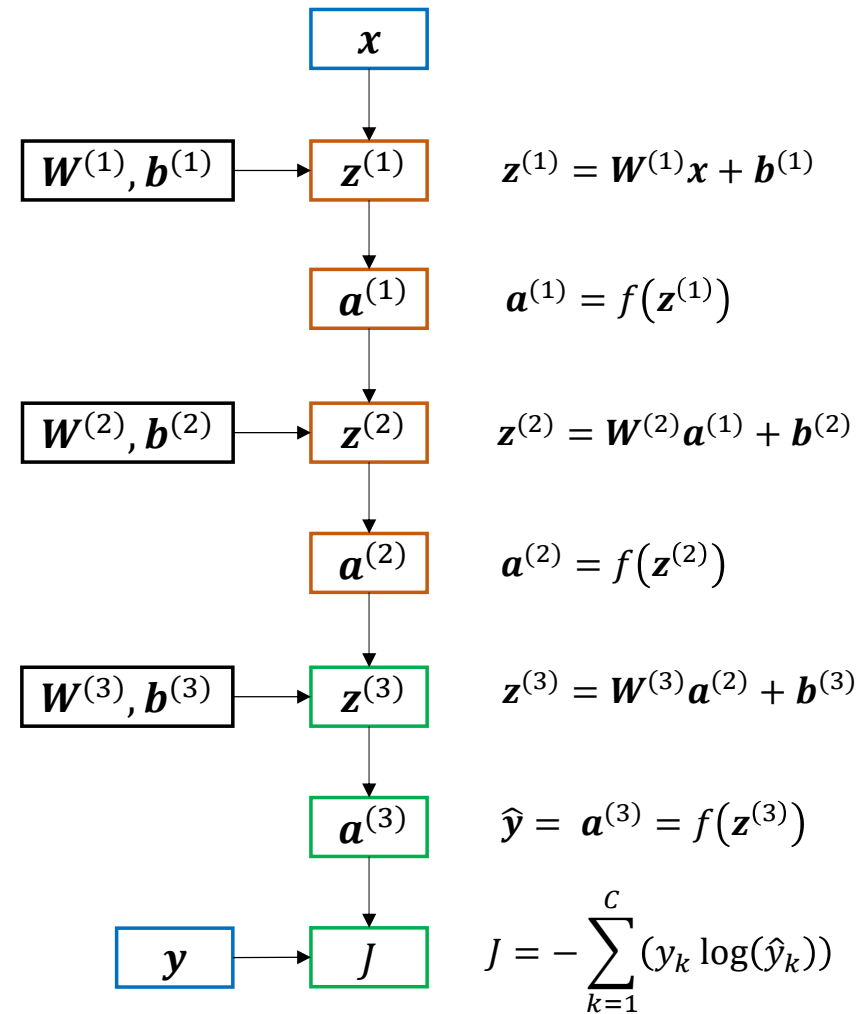


$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \text{ and } \mathbf{a}^{(1)} = f(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} \text{ and } \mathbf{a}^{(2)} = f(\mathbf{z}^{(2)})$$

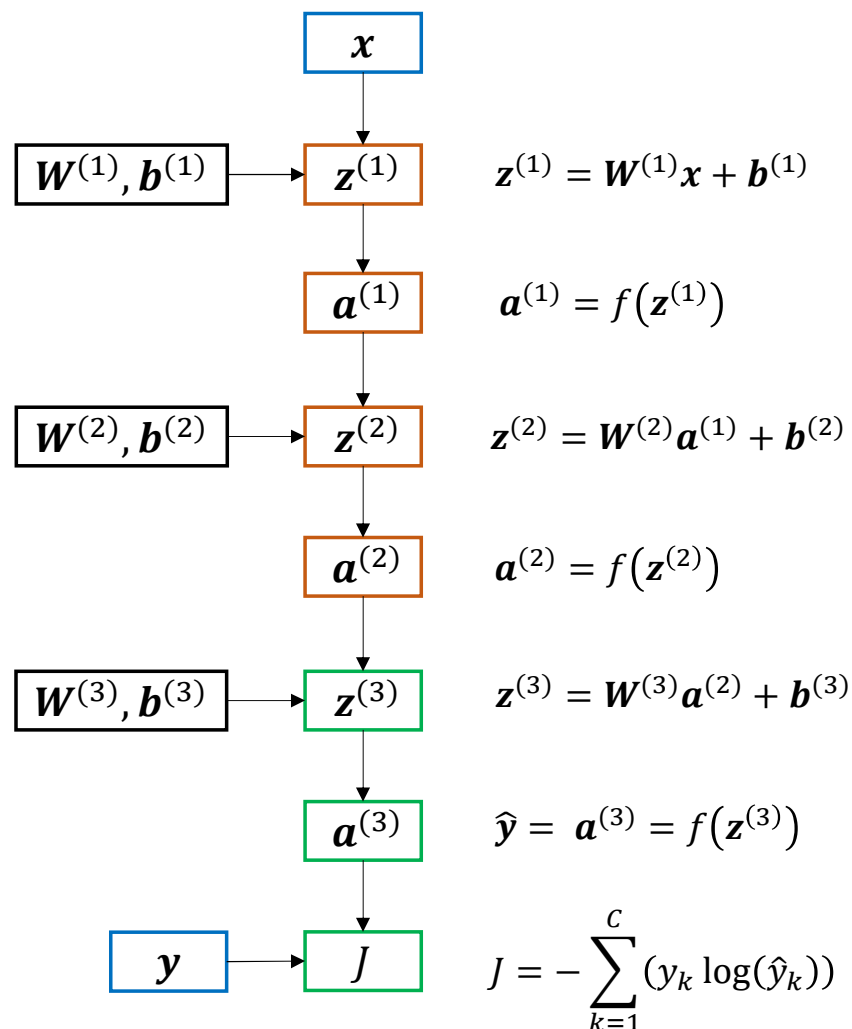
$$\mathbf{z}^{(3)} = \mathbf{W}^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)} \text{ and } \mathbf{a}^{(3)} = f(\mathbf{z}^{(3)})$$

The predicted output, $\hat{\mathbf{y}} = \mathbf{a}^{(3)}$



Back Propagation Learning Algorithm

From the computation graph we observe that:



Cost function J is a function of $\mathbf{a}^{(3)}$, $\mathbf{a}^{(3)}$ is in turn a function of $\mathbf{z}^{(3)}$. $\mathbf{z}^{(3)}$ is function of $\mathbf{W}^{(3)}$, $\mathbf{b}^{(3)}$ and $\mathbf{a}^{(2)}$.

So, to calculate the partial derivatives of the cost function J w.r.t. $\mathbf{W}^{(3)}$, $\mathbf{b}^{(3)}$ we will have to use the **chain rule**.

$$\frac{\partial J}{\partial \mathbf{W}^{(3)}} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{W}^{(3)}} \quad \text{and} \quad \frac{\partial J}{\partial \mathbf{b}^{(3)}} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{b}^{(3)}}$$

Similarly, to calculate the partial derivatives of the cost function J w.r.t. $\mathbf{W}^{(2)}$, $\mathbf{b}^{(2)}$ we will use the following formula:

$$\frac{\partial J}{\partial \mathbf{W}^{(2)}} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}} \cdot \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{W}^{(2)}}$$

and

$$\frac{\partial J}{\partial \mathbf{b}^{(2)}} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}} \cdot \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{b}^{(2)}}$$

Likewise, for calculating the partial derivatives of the cost function J w.r.t. $\mathbf{W}^{(1)}$, $\mathbf{b}^{(1)}$ we will use:

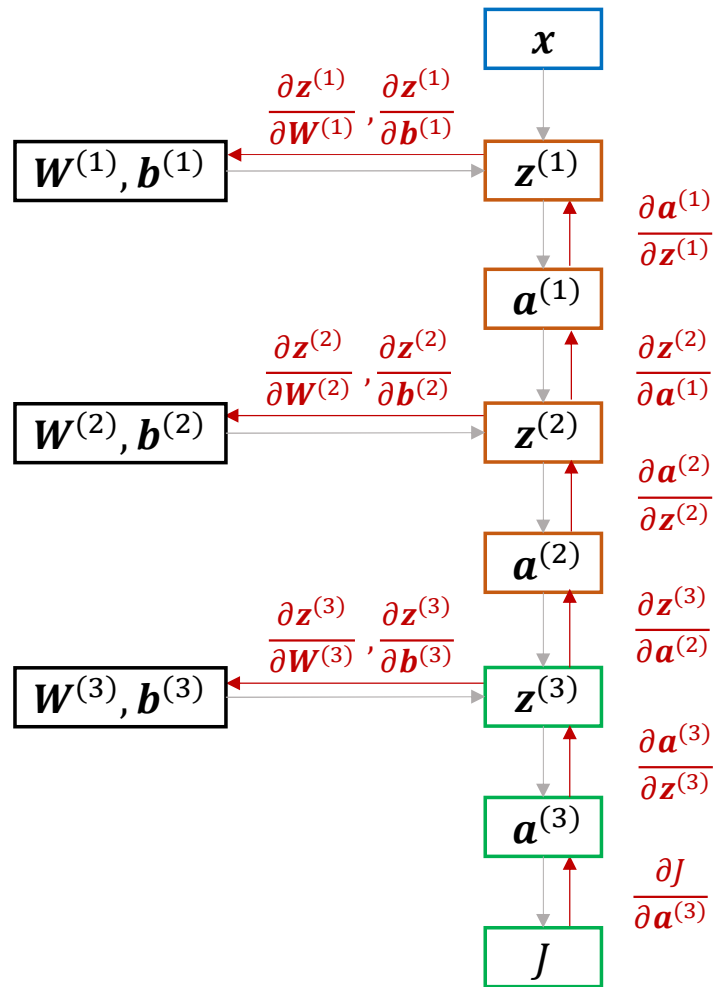
$$\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}} \cdot \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

and

$$\frac{\partial J}{\partial \mathbf{b}^{(1)}} = \frac{\partial J}{\partial \mathbf{a}^{(3)}} \cdot \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{z}^{(3)}} \cdot \frac{\partial \mathbf{z}^{(3)}}{\partial \mathbf{a}^{(2)}} \cdot \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{b}^{(1)}}$$

Back Propagation Learning Algorithm

This can be better understood from the following figure:



$$z^{(2)} = W^{(1)}x + b^{(1)}$$

$$a^{(2)} = f(z^{(2)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$$

$$a^{(2)} = f(z^{(2)})$$

$$z^{(3)} = W^{(3)}a^{(2)} + b^{(3)}$$

$$\hat{y} = a^{(3)} = f(z^{(3)})$$

$$J = - \sum_{k=1}^c (y_k \log(\hat{y}_k))$$

Forward propagation
Back propagation

- It's called **backpropagation** because the error (loss gradient) is **propagated backward** from the output layer to the input layer.
- This backward flow of gradients updates weights layer by layer using the chain rule.

Back Propagation Learning Algorithm

Input to the algorithm: Labelled dataset: $\langle X, y \rangle$; **Output of the algorithm:** Trained weights and biases

1. For each epoch : (Number of epochs should be specified by user)
2. Perform a feedforward pass, computing the activations for layers L_2, L_3 , and so on up to the output layer L_{n_l} .

3. For each output unit i in layer n_l (the final/output layer), set

$$\delta_i^{(n_l)} = \frac{\partial J}{\partial a_i^{(n_l)}} \cdot f' \left(z_i^{(n_l)} \right)$$

4. For $l = (n_l - 1), (n_l - 2), (n_l - 3), \dots, 2, 1$

For each node i in layer l , set

$$\delta_i^{(l)} = \left(\sum_{j=1}^{s_{l+1}} W_{ji}^{(l)} \delta_j^{(l+1)} \right) \cdot f' \left(z_i^{(l)} \right)$$

5. Compute the desired partial derivatives which are given as:

$$\frac{\partial}{\partial W_{ij}^{(l)}} J(W, b) = a_j^{(l)} \cdot \delta_i^{(l+1)} \quad \text{and} \quad \frac{\partial}{\partial b_i^{(l)}} J(W, b) = \delta_i^{(l+1)}$$

6. Finally update the parameters:

$$W_{ij}^{(l)} \leftarrow W_{ij}^{(l)} - \eta \frac{\partial}{\partial W_{ij}^{(l)}} J(W, b) \quad \text{and} \quad b_i^{(l)} \leftarrow b_i^{(l)} - \eta \frac{\partial}{\partial b_i^{(l)}} J(W, b); \quad \eta \text{ is the learning rate}$$

Thank You